Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5
По дисциплине: «Интеллектуальный анализ данных»
Тема: «Деревья решений»

Выполнил:
Студент 4 курса
Группы ИИ-26
Суховило В. С.
Проверил:
Никитин С. К.

Брест 2026

Цель работы:

На практике сравнить работу одиночного дерева решений, случайного леса и алгоритмов бустинга для решающих деревьев.

Общее задание:

1. Загрузить Mushroom Classification и закодировать категориальные признаки методом One-Hot Encoding.

2. Разделить набор на обучающую и тестовую части, обучить дерево решений, случайный лес, AdaBoost, CatBoost, XGBoost.

3. Оценить precision и recall по классу ядовитых грибов, сравнить ошибки и сделать выводы.

Вариант №5. Исходные данные:

UCI Mushroom Classification: 8124 записи, категориальные признаки; класс p = ядовитый, e = съедобный.

Реализация и методика эксперимента:

Stratified train/test split 75/25, OneHotEncoder обучается на тренировочных признаках; далее независимо обучаются пять моделей. Класс poisonous кодируется единицей. Метрики считаются только на тестовой части.

Результат программы:

Метод	Accuracy	Precision p	Recall p	FN
Decision Tree	1.0000	1.0000	1.0000	0
Random Forest	1.0000	1.0000	1.0000	0
AdaBoost	0.9985	1.0000	0.9969	3
CatBoost	1.0000	1.0000	1.0000	0
XGBoost	1.0000	1.0000	1.0000	0

Графики и иллюстрации:

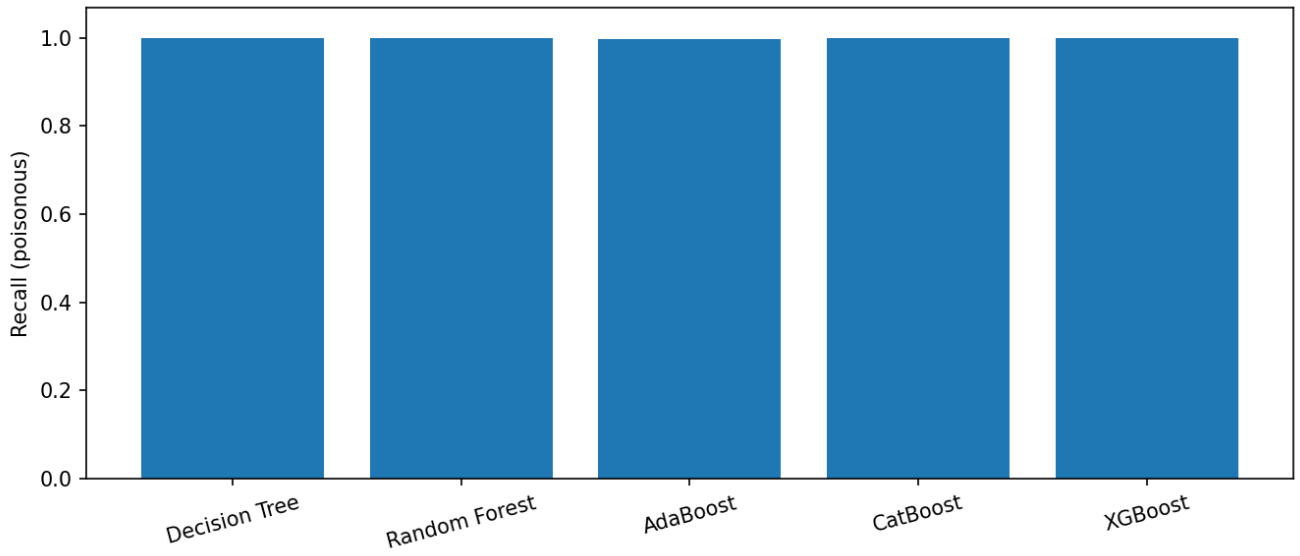


Рисунок. model comparison

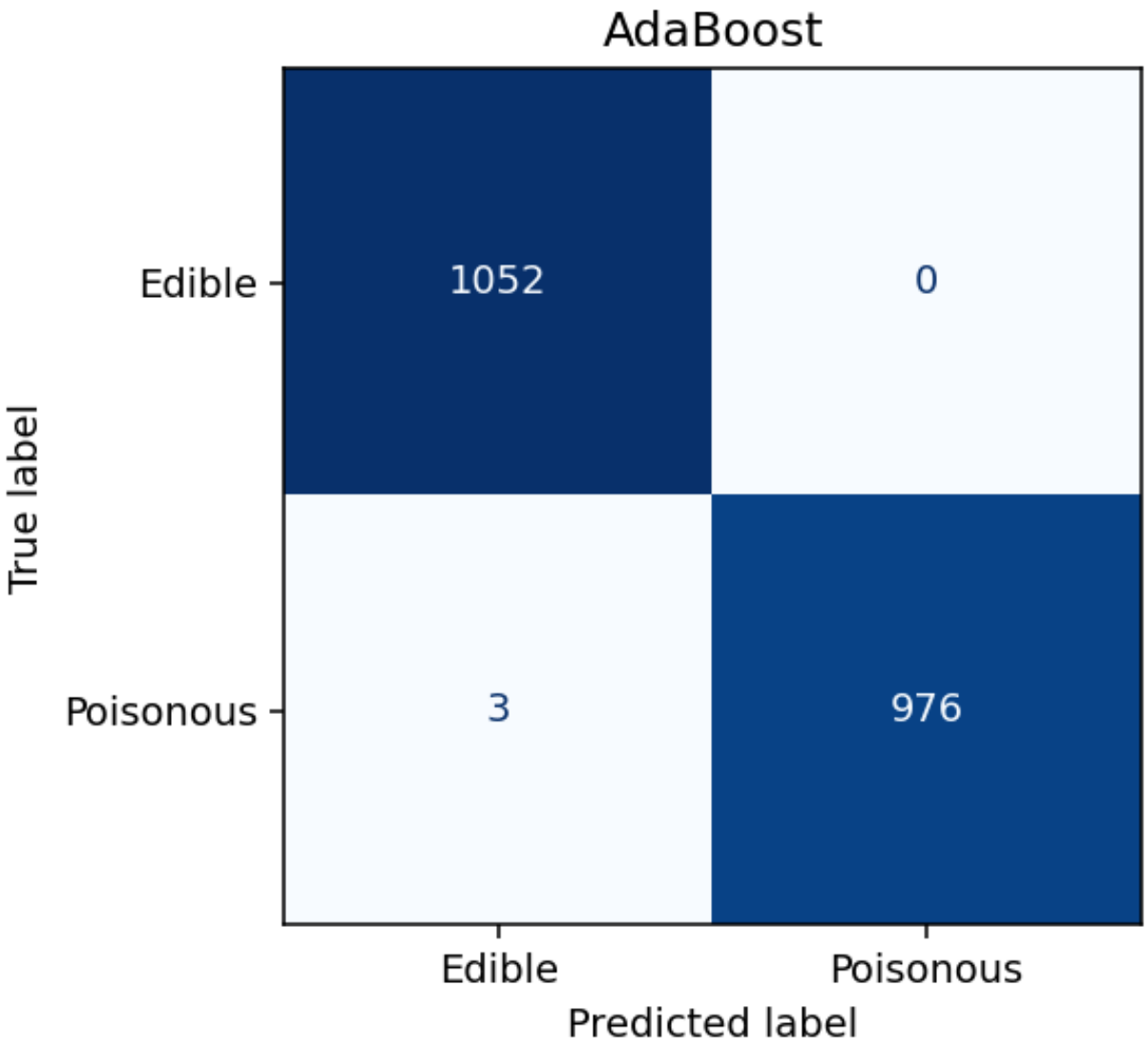


Рисунок. confusion adaboost

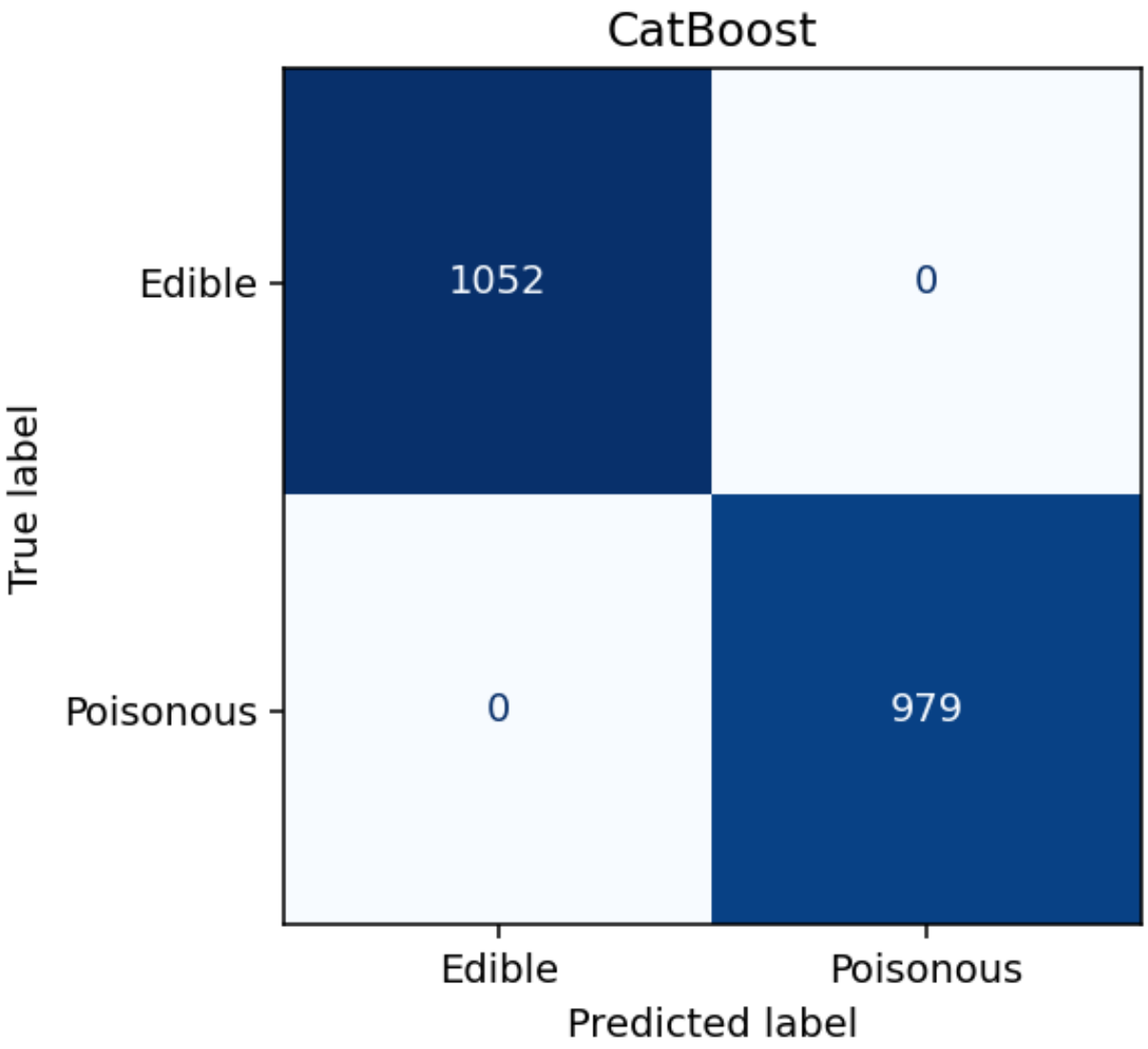


Рисунок. confusion catboost

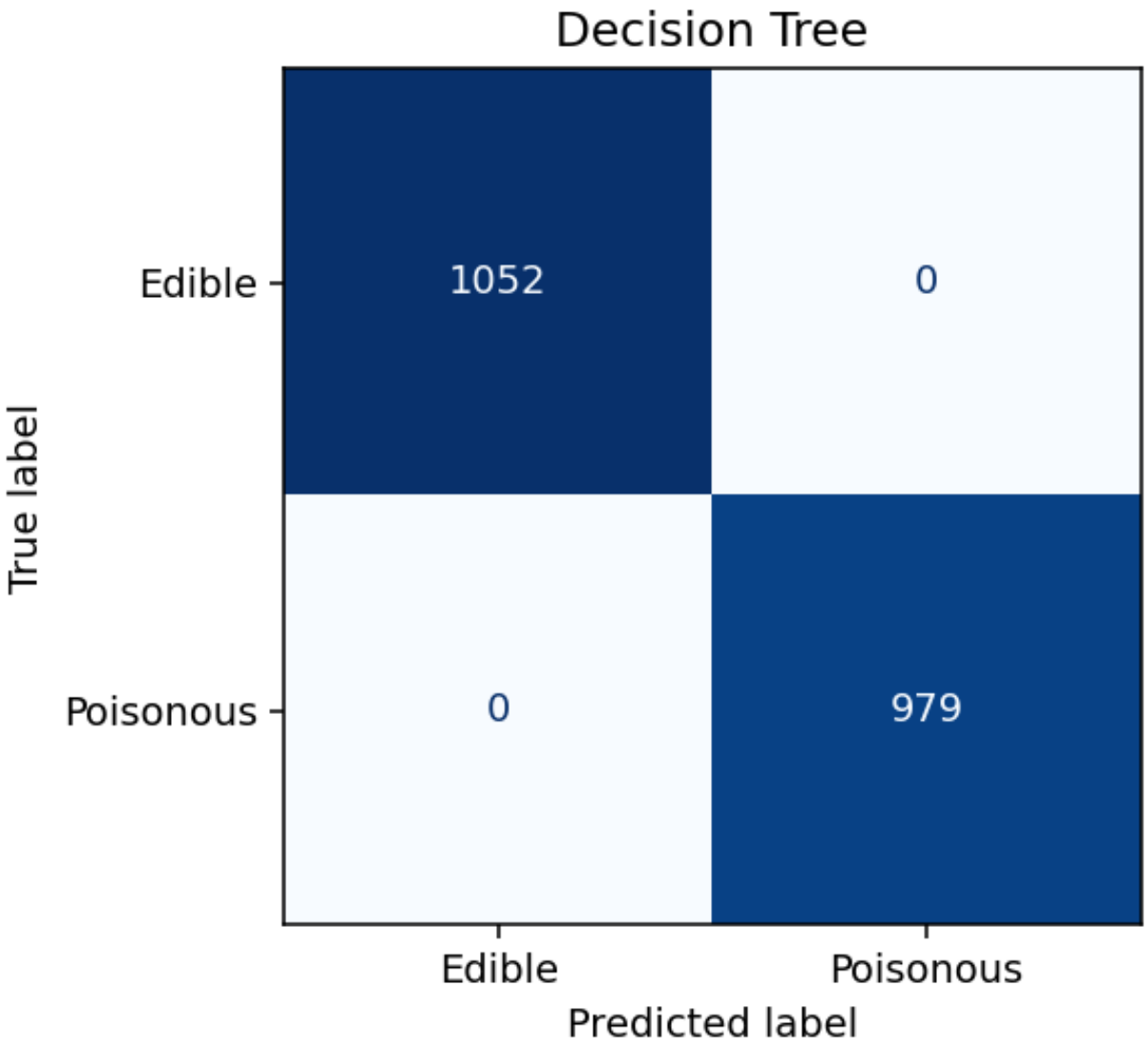


Рисунок. confusion decision tree

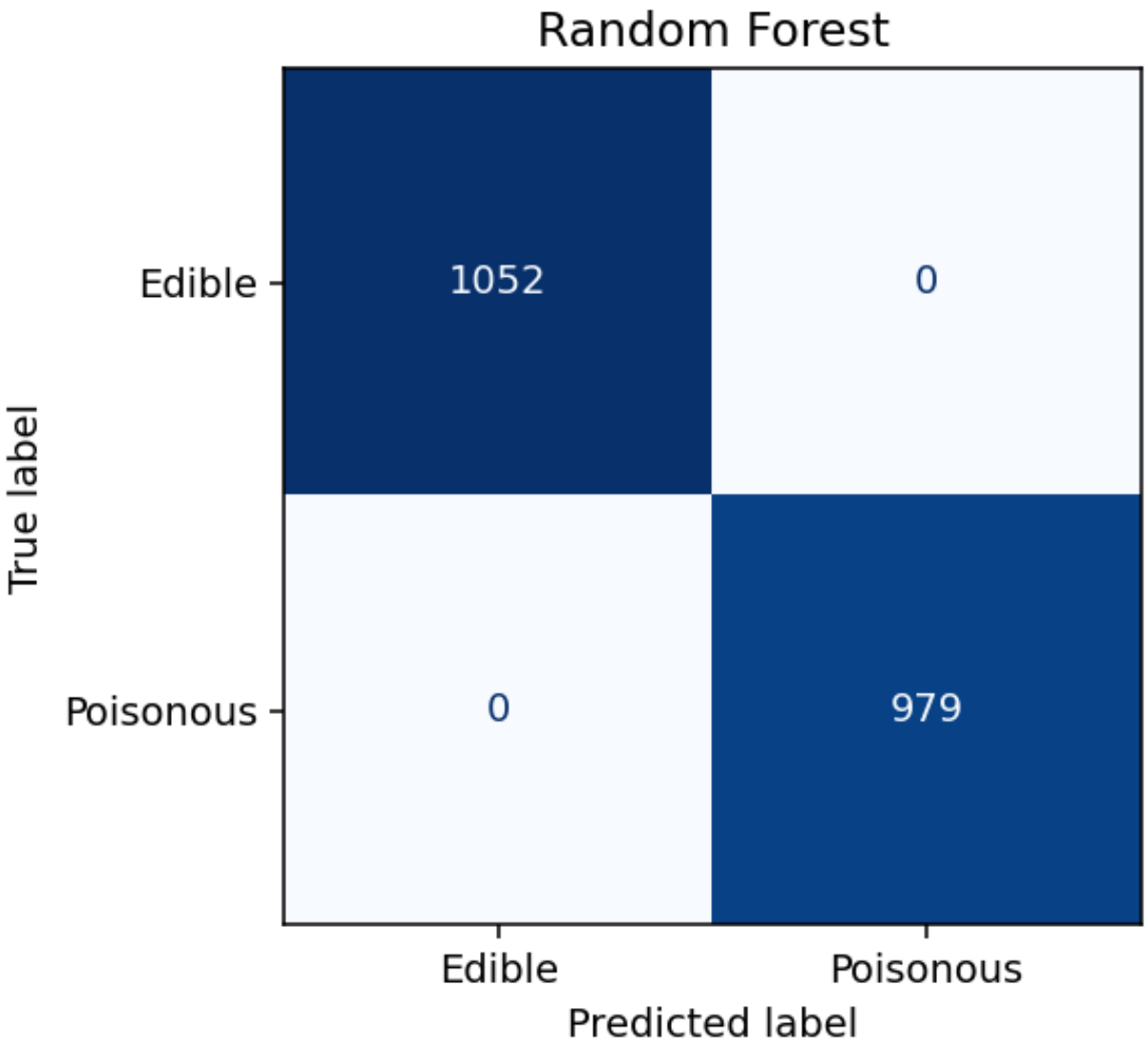


Рисунок. confusion random forest

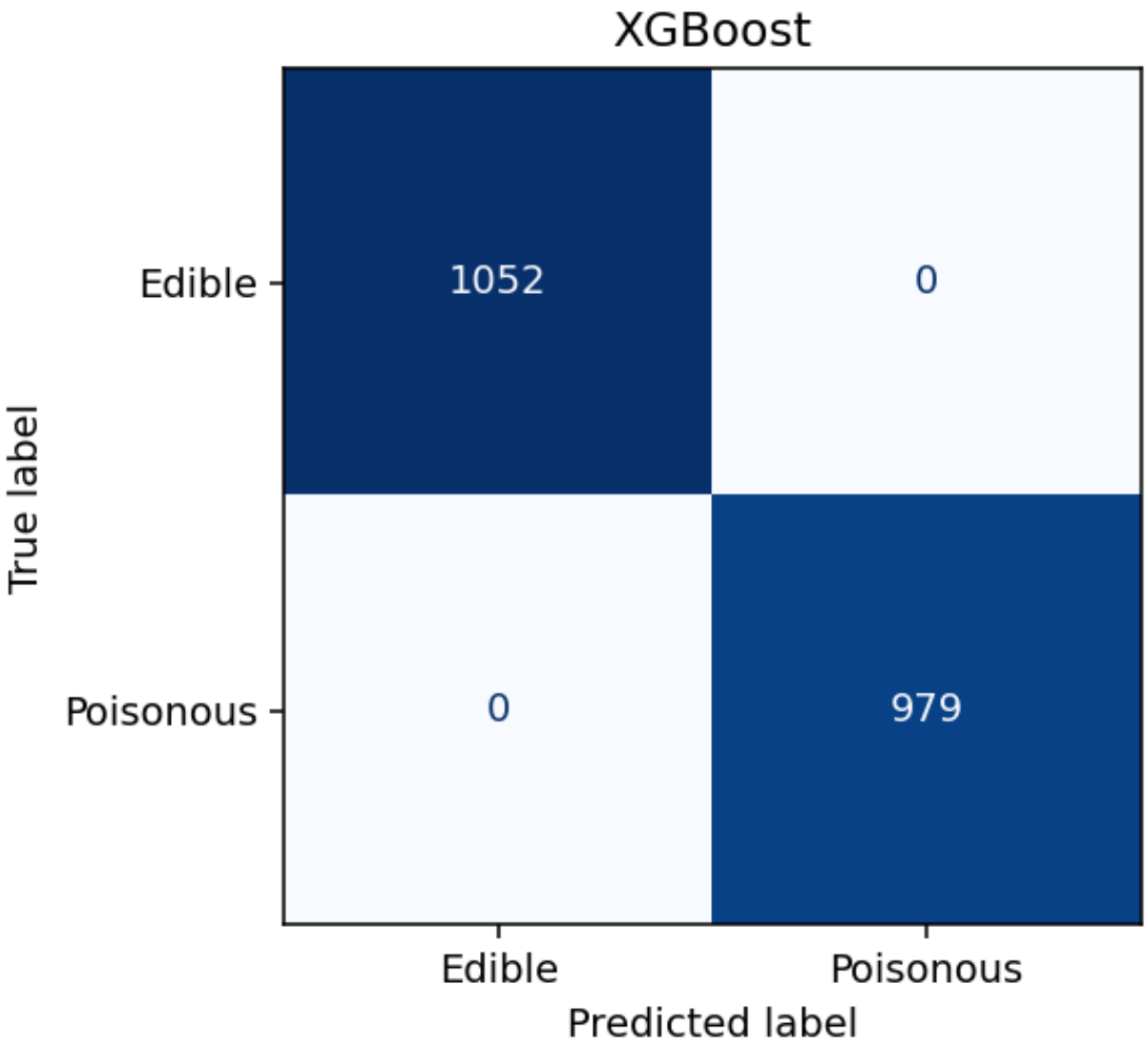


Рисунок. confusion xgboost

Выводы о проделанной работе:

1. Decision Tree: accuracy=1.0000, precision(ядовитый)=1.0000, recall(ядовитый)=1.0000, FN=0.
2. Random Forest: accuracy=1.0000, precision(ядовитый)=1.0000, recall(ядовитый)=1.0000, FN=0.
3. AdaBoost: accuracy=0.9985, precision(ядовитый)=1.0000, recall(ядовитый)=0.9969, FN=3.
4. CatBoost: accuracy=1.0000, precision(ядовитый)=1.0000, recall(ядовитый)=1.0000, FN=0.
5. XGBoost: accuracy=1.0000, precision(ядовитый)=1.0000, recall(ядовитый)=1.0000, FN=0.
6. Цена ложноотрицательной ошибки при распознавании ядовитого гриба особенно высока, поэтому основной ориентир — recall для класса poisonous и число FN.
7. Идеальные результаты нескольких моделей относятся только к данному учебному набору и выбранному тестовому разбиению; их нельзя переносить на распознавание реальных грибов.

Код программы: ИАД_5.py

```
"""ИАД ЛР №5 вариант 5: классификация грибов пятью деревообразными моделями."""
from pathlib import Path
import json,time
import numpy as np,pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score,confusion_matrix,ConfusionMatrixDisplay
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier,AdaBoostClassifier
from xgboost import XGBClassifier
from catboost import CatBoostClassifier
ROOT=Path(__file__).resolve().parent;OUT=ROOT/'results';OUT.mkdir(exist_ok=True)
df=pd.read_csv(ROOT/'data/mushrooms.csv');target=(df['class']=='p').astype(int)
features=df.drop(columns=['class'])
# '?' = реальное неизвестное значение в отдельных категориальных признаках
X_tr,X_te,y_tr,y_te=train_test_split(features,target,test_size=.25,random_state=42,stratify=target)
encoder=OneHotEncoder(handle_unknown='ignore',sparse_output=False)
Xtrain=encoder.fit_transform(X_tr);Xtest=encoder.transform(X_te)
models={
 'Decision Tree':DecisionTreeClassifier(random_state=42,max_depth=12),
 'Random Forest':RandomForestClassifier(n_estimators=120,random_state=42,n_jobs=2),
 'AdaBoost':AdaBoostClassifier(n_estimators=120,learning_rate=.7,random_state=42),
 'CatBoost':CatBoostClassifier(iterations=160,learning_rate=.09,depth=5,random_seed=42,verbose=False,thread_count=2),
 'XGBoost':XGBClassifier(n_estimators=160,max_depth=5,learning_rate=.09,random_state=42,eval_metric='logloss',n_jobs=2)
}
metrics={}
for name,model in models.items():
 start=time.perf_counter();model.fit(Xtrain,y_tr);timing=time.perf_counter()-start
 pred=model.predict(Xtest)

metrics[name]={'accuracy':float(accuracy_score(y_te,pred)),'precision_poisonous':float(precision_score(y_te,pred,zero_division
=0)),
  'recall_poisonous':float(recall_score(y_te,pred,zero_division=0)),'f1_poisonous':float(f1_score(y_te,pred,zero_division=0)),

'false_negatives':int(((y_te==1)&(pred==0)).sum()),'confusion_matrix':confusion_matrix(y_te,pred).tolist(),'training_seconds':
round(timing,3)}

fig,ax=plt.subplots(figsize=(4.5,4));ConfusionMatrixDisplay.from_predictions(y_te,pred,display_labels=['Edible','Poisonous'],a
x=ax,colorbar=False,cmap='Blues');ax.set_title(name);fig.tight_layout();fig.savefig(OUT/f'confusion_{name.lower().replace("
","_")}.png',dpi=140);plt.close(fig)
 print(name,metrics[name])
(OUT/'metrics.json').write_text(json.dumps({'dataset_rows':len(df),'train_size':len(y_tr),'test_size':len(y_te),'encoded_featu
res':Xtrain.shape[1],'models':metrics},indent=2,ensure_ascii=False),encoding='utf-8')
fig,ax=plt.subplots(figsize=(9,4));names=list(metrics);vals=[metrics[n]['recall_poisonous'] for n in
names];ax.bar(names,vals);ax.set_ylim(0,1.07);ax.set_ylabel('Recall
(poisonous)');ax.tick_params(axis='x',labelrotation=15);fig.tight_layout();fig.savefig(OUT/'model_comparison.png',dpi=150);plt
.close(fig)
```

Исходный код, данные и графики также приложены к лабораторной работе в директории src/.